

Lean 与 AI

从 instance 到可检查的形式化代码

今天抓住一条主线：结构实例、Mathlib 检索、AI 草稿怎样一起变成可检查证明。

instance 是 Lean 使用数学结构的入口

对象

``G : Type*``
只是一个类型

结构

``[Group G]``
要求 Lean 找到群结构

定理

``mul_assoc``
可在任何半群上使用

```
variable (G : Type*) [Group G]
```

```
#synth Group G  
#check mul_assoc  
#check inv_mul_cancel
```

写 instance 时, Lean 不接受口头承诺

要给数据

- ``mul`, `one`, `inv``
- ``add`, `zero`, `neg``
- ``Hom`, `id`, `comp``
- 这些决定记号怎么解释

要给证明

- 结合律、单位律、逆元律
- 分配律、交换律
- 范畴的三条公理
- 每一条都是一个 goal

检查方法: ``#print`` 看字段; 在字段后写 ``:= by`` 看目标; 逐条完成。

AddGroup 是 Group 的加法语言

结构	使用的符号	核心数据	核心公理
`AddGroup A`	`0`, `+`, `-a`	`zero`, `add`, `neg`	加法结合律、零元律、加法逆元律
`Group G`	`1`, `*`, `g <sup>-1</sup> `	`one`, `mul`, `inv`	乘法结合律、单位元律、乘法逆元律

```
#check AddGroup
#check Group
#check neg_add_cancel
#check inv_mul_cancel

example (A : Type*) [AddGroup A] (a : A) :
  -a + a = 0 := neg_add_cancel a

example (G : Type*) [Group G] (g : G) :
  g-1 * g = 1 := inv_mul_cancel g
```

讲清这点很重要

- 它们不是同一个 class
- 数学模式相同
- Lean 根据符号找接口
- 加法结构常用于环和模

AddGroup 出现在环和模的底层

群论语言

- `[Group G]` 给乘法群
- 定理写成 ``a * b``
- ``inv_mul_cancel`` 讲逆元
- 非交换时顺序很敏感

线性代数语言

- `[AddCommGroup V]` 给向量加法
- 定理写成 ``u + v``
- ``neg_add_cancel`` 讲相反元
- `[Module K V]` 在其上加标量作用

```
#check add_assoc    -- additive interface
#check mul_assoc    -- multiplicative interface
#check neg_add_cancel -- AddGroup
#check inv_mul_cancel -- Group
```

Group 要检查四类条件

字段	数学意义	常见证明方式
<code>`mul`</code>	乘法运算 $G \rightarrow G \rightarrow G$	给定义，有限类型可分情况
<code>`one`</code>	单位元	给一个元素
<code>`inv`</code>	逆元函数	给 $G \rightarrow G$
<code>`mul_assoc`</code>	乘法结合律	<code>`cases`</code> / 已知定理 / <code>`group`</code>
<code>`one_mul`</code> , <code>`mul_one`</code>	左右单位律	化简或直接 <code>`rfl`</code>
<code>`inv_mul_cancel`</code>	左逆元律	分情况或调用定理

例子：两元素 Sign 群

```
inductive Sign where
  | pos
  | neg
deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
  one := pos
  inv a := a
  -- laws come next
```

- `pos` 是单位元
- `neg \* neg = pos`
- 每个元素都是自己的逆元
- 接下来逐条证明公理

教学动作：先让学生预测 `pos \* neg` 和 `neg \* neg`。

Group 公理逐条检查

```
mul_assoc := by
  intro a b c
  cases a <;> cases b <;> cases c <;> rfl
```

```
one_mul := by
  intro a
  cases a <;> rfl
```

```
mul_one := by
  intro a
  cases a <;> rfl
```

```
inv_mul_cancel := by
  intro a
  cases a <;> rfl
```

1. ``intro`` 把任意元素放进上下文
2. ``cases`` 枚举 ``pos`` 和 ``neg``
3. ``rfl`` 检查定义化简后两边相同
4. 四个字段缺一个都不是 Group

```
#synth Group Sign
example (a : Sign) : a-1 * a = 1 := by
  exact inv_mul_cancel a
```


Subtype : 一个元素可以同时携带性质证明

```
def NonzeroInt := {n :  $\mathbb{Z}$  // n  $\neq$  0}

example : NonzeroInt :=
  {1, by norm_num}

example (x : NonzeroInt) :  $\mathbb{Z}$  :=
  x.1

example (x : NonzeroInt) : x.1  $\neq$  0 :=
  x.property

#check Subtype.val
#check Subtype.property
```

读法

- ``x.1`` 是底层值
- ``x.2`` 或 ``x.property`` 是证明
- ``(value, proof)`` 构造元素
- 子群、子环、子模都沿用这个思想

Subtype 不是“新集合”这么简单；它让元素和值满足的性质绑定在一起。

Subgroup = carrier 谓词 + 群运算封闭性

```
def oneSubgroup (G : Type*) [Group G] :  
  Subgroup G where  
  carrier := {g | g = 1}  
  one_mem' := by  
    rfl  
  mul_mem' := by  
    intro a b ha hb  
    rw [ha, hb]  
    simp  
  inv_mem' := by  
    intro a ha  
    rw [ha]  
    simp
```

字段逐条检查

1. `carrier` : 哪些 `g : G` 属于子群
2. `one\_mem` : 单位元属于它
3. `mul\_mem` : 乘法封闭
4. `inv\_mem` : 逆元封闭
5. Mathlib 给子类型上的群结构

```
example (G : Type*) [Group G] (x : oneSubgroup G) :  
  (x : G) = 1 := x.property
```

Ring 条件更多，因为它同时有加法和乘法

部分	要满足什么	Lean 中常见入口
加法	交换群：`0`, `+`, `-`, 加法结合 / 交换 / 逆元	`AddCommGroup`
乘法	么半群：`1`, `*`, 乘法结合 / 单位元	`Monoid`
相互作用	左右分配律，`0` 乘任何元素为 `0`	`left_distrib`, `right_distrib`
交换环	再加乘法交换律	`CommRing` / `mul_comm`

``Ring`` 的完整字段很多，课堂重点是：每个符号对应数据，每条代数律对应证明。

Ring instance 常用策略：沿等价迁移结构

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt  $\simeq$  Int  $\times$  Int where
  toFun p := (p.x, p.y)
  invFun q := (q.1, q.2)
  left_inv := by intro p; ext <;> rfl
  right_inv := by intro q; cases q; rfl

instance : CommRing PairInt :=
  equivProd.commRing
```

- ``Int  $\times$  Int`` 已经有交换环结构
- ``PairInt`` 与它等价
- ``Equiv.commRing`` 负责迁移
- 之后仍可逐条检查环定理

Ring 条件也可以逐条检查

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm

example (p q r : PairInt) :
  p * (q + r) = p * q + p * r := by
  exact mul_add p q r

example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2 := by
  ring
```

检查顺序

1. `#synth`` 确认实例存在
2. `#check`` 看定理需要什么结构
3. 用小定理验证字段效果
4. `ring`` 处理多项式整理

Subring 比 Subgroup 多检查加法和乘法封闭

```
def diagonalSubring : Subring ( $\mathbb{Z} \times \mathbb{Z}$ ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb
    rw [ha, hb]
  neg_mem' := by
    intro a ha
    simp at ha
    rw [ha]
  mul_mem' := by
    intro a b ha hb
    simp at ha hb
    rw [ha, hb]
```

对角子环

- 元素是满足 `p.1 = p.2` 的 pair
- `zero\_mem` 和 `one\_mem` 给常元
- `add\_mem`, `neg\_mem`, `mul\_mem` 给封闭性
- 元素有 `x.property`
- 子类型自动有环结构

```
example (x : diagonalSubring) :
  (x :  $\mathbb{Z} \times \mathbb{Z}$ ).1 = (x :  $\mathbb{Z} \times \mathbb{Z}$ ).2 := by
  exact x.property
```

Field = 交换环 + 非零元素可逆

条件	数学含义	Lean 里常见表现
<code>`CommRing K`</code>	先是交换环	<code>`ring`</code> 可用
<code>`Nontrivial K`</code>	至少有两个元素	<code>`0 ≠ 1`</code>
<code>`inv`, `div`</code>	逆元和除法	<code>`x / y = x * y<sup>-1`</sup></code>
<code>`mul_inv_cancel`</code>	非零元素有乘法逆元	需要假设 <code>`x ≠ 0`</code>
<code>`inv_zero`</code>	Lean 中约定 <code>`0<sup>-1</sup> = 0`</code>	让逆元函数全定义

最常见 AI 错误：把 ``x / x = 1`` 写成无条件命题。

例子：包装 Rat 得到一个 Field

```
@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat where
  toFun x := x.val
  invFun q := { val := q }
  left_inv := by intro x; ext; rfl
  right_inv := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field
```

- 结构来自已知域 `Rat`
- 实例可由等价自动迁移
- `field\_simp [hx]` 使用非零假设
- 这里的重点是 statement

Module 是向量空间的 Lean 接口

组成	数学含义	Lean 中的假设
标量	可以相加相乘的系数	<code>`[Semiring R]`</code> 或 <code>`[Field K]`</code>
向量	可以相加取相反数	<code>`[AddCommGroup V]`</code>
标量作用	$r \bullet v$	<code>`[Module R V]`</code>
兼容律	分配律、结合律、单位元律	<code>`smul_add`</code> , <code>`add_smul`</code> , <code>`mul_smul`</code> , <code>`one_smul`</code>

```
variable {K : Type*} [Field K]
variable {V : Type*} [AddCommGroup V] [Module K V]
```

```
#check smul_add
#check add_smul
#check mul_smul
#check one_smul
```

Submodule 是 subtype 加线性封闭性

```
def diagonalSubmodule : Submodule  $\mathbb{Q}$  ( $\mathbb{Q} \times \mathbb{Q}$ ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by
    simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb
    rw [ha, hb]
  smul_mem' := by
    intro c p hp
    change (c * p.1 = c * p.2)
    rw [hp]
```

要检查什么

1. `carrier` : 哪些向量属于子空间
2. `zero\_mem` : 零向量在里面
3. `add\_mem` : 加法封闭
4. `smul\_mem` : 数乘封闭
5. 子类型自动有模结构

```
example (x : diagonalSubmodule) :
  (x :  $\mathbb{Q} \times \mathbb{Q}$ ).1 = (x :  $\mathbb{Q} \times \mathbb{Q}$ ).2 := x.property
```

向量空间例子把存在性证明变成可用函数

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module F W]
variable [FiniteDimensional F W]

def IsAlternating (ω : W →[F] W →[F] F) : Prop :=
  ∀ w : W, ω w w = 0

def IsNondegenerate (ω : W →[F] W →[F] F) : Prop :=
  ∀ w : W, (∀ v : W, ω w v = 0) → w = 0
```

建模读法

- `W` 是向量空间
- `ω` 是双线性型
- `Prop` 存数学性质
- 先表达对象和性质

这类例子适合让 AI 先写结构草稿，但每个存在性声明都必须由 Lean 证明或作为字段保存。

CompletePolarization 把分解假设保存成字段

```
def TotallyIsotropic
  (ω : W →[F] W →[F] F) (U : Submodule F W) : Prop :=
  ∀ u : W, u ∈ U → ∀ v : W, v ∈ U → ω u v = 0

structure CompletePolarization
  (ω : W →[F] W →[F] F) where
  X : Submodule F W
  Y : Submodule F W
  isotropic_X : TotallyIsotropic F W ω X
  isotropic_Y : TotallyIsotropic F W ω Y
  disjoint : Disjoint X Y
  sup_eq_top : X ⊔ Y = ⊤
```

字段含义

- `X`, `Y` 是子空间
- 分别全迷向
- `Disjoint X Y`
- `X ⊔ Y = ⊤`

`sup\_eq\_top` 是后面分解任意 `w : W` 的关键入口。

Classical.choose 从存在性证明中取一个对象

```
#check Classical.choose
-- Classical.choose :
-- (h :  $\exists x, p\ x$ )  $\rightarrow \alpha$ 

#check Classical.choose_spec
-- Classical.choose_spec :
-- p (Classical.choose h)

lemma exists_decomposition (P : CompletePolarization F W  $\omega$ ) (w : W) :
   $\exists x : W, x \in P.X \wedge \exists y : W, y \in P.Y \wedge w = x + y :=$  by
  ...

noncomputable def xComponent (w : W) : W :=
  Classical.choose (exists_decomposition F W P w)
```

怎么讲

- 证明 $\exists x, p\ x$
- `choose` 取 witness
- `choose\_spec` 取性质
- 通常 `noncomputable`

选择公理允许“选一个满足条件的对象”，但不告诉你可计算算法。

choose_spec 负责证明选出来的对象真的满足条件

```

noncomputable def yComponent
  (P : CompletePolarization F W ω) (w : W) : W :=
  Classical.choose
  ((Classical.choose_spec (exists_decomposition F W P w)).2)

lemma decompose_by_components
  (P : CompletePolarization F W ω) (w : W) :
  w = xComponent F W ω P w + yComponent F W ω P w := by
  dsimp [xComponent, yComponent]
  exact
  (Classical.choose_spec
    ((Classical.choose_spec
      (exists_decomposition F W P w)).2)).2

```

嵌套选择

- 第一层选 ``x``
- 证明里还含 ``∃ y``
- 第二层再选 ``y``
- 最后取等式 ``w = x + y``

Lean 不会忘记 witness 的性质；只是你要用 ``choose_spec`` 明确把证明取出来。

Category 的定义分成数据和公理

```
class CategoryStruct (obj : Type u) extends Quiver obj where
  id :  $\forall X : \text{obj}, \text{Hom } X X$ 
  comp :  $\forall \{X Y Z : \text{obj}\},$ 
     $(X \longrightarrow Y) \rightarrow (Y \longrightarrow Z) \rightarrow (X \longrightarrow Z)$ 
```

```
class Category (obj : Type u) extends CategoryStruct obj where
  id_comp :  $\forall \{X Y\} (f : X \longrightarrow Y), \mathbb{1} X \gg f = f$ 
  comp_id :  $\forall \{X Y\} (f : X \longrightarrow Y), f \gg \mathbb{1} Y = f$ 
  assoc :  $\forall f g h, (f \gg g) \gg h = f \gg g \gg h$ 
```

读定义

- ``Hom`` 来自 ``Quiver``
- `` $\mathbb{1} X$ `` 是 ``id X``
- ``f >> g`` 是复合
- instance 时要给数据
- 再逐条证明三条律

示意化摘录：为课堂阅读省略 universe 与部分字段，完整可运行代码见 Lean 文件。

Functor 保存对象像、态射像和两条保持律

```
structure Functor (C : Type u1) [Category C]
  (D : Type u2) [Category D] where
  obj : C → D
  map : ∀ {X Y : C}, (X → Y) → (obj X → obj Y)
  map_id : ∀ X, map (1 X) = 1 (obj X)
  map_comp : ∀ f g, map (f >> g) = map f >> map g
```

- ``F.obj X`` : 对象的像
- ``F.map f`` : 态射的像
- ``F >>> G`` : 函子复合

``F : C ⇒ D`` 不是普通函数；它同时搬运对象、态射，并证明保单位和保复合。

示意化摘录：为课堂阅读省略 universe 与部分字段，完整可运行代码见 Lean 文件。

NatTrans 是分量态射加自然性方块

```
structure NatTrans (F G : C  $\Rightarrow$  D) where
  app (X : C) : F.obj X  $\longrightarrow$  G.obj X
  naturality {X Y : C} (f : X  $\longrightarrow$  Y) :
    F.map f  $\gg$  app Y = app X  $\gg$  G.map f
```

课堂读法

- 分量：`η.app X`
- 左右两路相同
- 自然性即交换方块

在 Mathlib 中，自然变换本身就是两个函子之间的 morphism，所以也写作 ` $\eta : F \longrightarrow G$ `。

示意化摘录：为课堂阅读省略 universe 与部分字段，完整可运行代码见 Lean 文件。

HomologicalComplex 把链复形写成结构字段

```
structure HomologicalComplex (c : ComplexShape ι) where
  X : ι → V
  d : ∀ i j, X i → X j
  shape : ∀ i j, ¬ c.Rel i j → d i j = 0
  d_comp_d' : ∀ i j k,
    c.Rel i j → c.Rel j k → d i j >> d j k = 0
```

字段含义

- `X i` 是第 `i` 项
- `d i j` 是微分
- `shape` : 非法箭头为零
- `d_comp_d'` : 复合为零

`ComplexShape.up ℕ` 给出向上编号的链形状；`HomologicalComplex.d_comp_d` 是库中包装好的连续微分为零定理。

示意化摘录：为课堂阅读省略 universe 与部分字段，完整可运行代码见 Lean 文件。

例子一：一个对象一个态射的 Category instance

```
open CategoryTheory

inductive OneObj where
| star

instance : Category OneObj where
  Hom __ := PUnit
  id _ := PUnit.unit
  comp __ := PUnit.unit
  id_comp := by
    intro X Y f
    cases f
    rfl
  comp_id := by
    intro X Y f
    cases f
    rfl
  assoc := by
    intro W X Y Z f g h
    cases f
    cases g
    cases h
    rfl
```

检查顺序

1. 给 `Hom/id/comp`
2. 证明左单位律
3. 证明右单位律
4. 证明结合律

例子二：零微分给出一个 HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits

noncomputable def tinyZeroComplex :
  HomologicalComplex (Discrete PUnit) (ComplexShape.up  $\mathbb{N}$ ) where
  X_ := Discrete.mk PUnit.unit
  d__ := 0
  shape := by
    intro i j hij
    rfl
  d_comp_d' := by
    intro i j k hij hjk
    simp
```

为什么成立？

- `Discrete PUnit` 很小
- 所有微分取 `0`
- `shape` 自动满足
- 复合为零：`simp`

这不是深同调代数定理，而是展示 Mathlib 的链复形接口可以被具体构造。

命题也可以作为 instance

普通结构 instance

- 通常在 `Type` 中
- 携带操作和数据
- 例: `Group G`, `Field K`
- 影响记号和可用定理

命题 instance

- 类本身在 `Prop` 中
- 携带可自动传递的证明
- 例: `Fact P`, `[Decidable P]`
- 常用于把假设放进搜索链

Fact P : 把一个命题放进 typeclass search

```
instance : Fact (2 ≤ 5) :=  
  ⟨by norm_num⟩  
  
example : 2 ≤ 5 := by  
  exact Fact.out  
  
example (n : Nat) [Fact (n ≠ 0)] :  
  0 < n := by  
  exact Nat.pos_of_ne_zero  
    (Fact.out : n ≠ 0)
```

1. `Fact P` 的字段就是 `P` 的证明
2. `Fact.out` 取出证明
3. `[Fact ...]` 可隐式传入
4. `have!` 可临时创建实例

```
have! : Fact (2 ≤ 5) := ⟨by norm_num⟩
```

自定义命题 class : 把数学性质交给搜索

```
class IsEven (n : Nat) : Prop where
  witness :  $\exists k, n = 2 * k$ 

instance : IsEven 8 where
  witness := ⟨4, by norm_num⟩

example [h : IsEven n] :
   $\exists k, n = 2 * k$  := by
  exact h.witness

example :  $\exists k, 8 = 2 * k$  := by
  exact (inferInstance : IsEven 8).witness
```

- 类名表达性质
- 实例表达某个对象满足性质
- ``inferInstance`` 让 Lean 搜索证明
- 适合局部、可复用的性质假设

instance search 的三个课堂工具

``#synth``

确认 Lean 能否找到指定实例

``infer_instance``

在 proof 中调用实例搜索

``have!``

临时把一个证明或结构放进搜索环境

```
#synth Group Sign
```

```
example : Group Sign := by  
  infer_instance
```

```
example : 2 ≤ 5 := by  
  have! : Fact (2 ≤ 5) := (by norm_num)  
  exact Fact.out
```


更深入：隐式参数决定定理能在哪些结构上用

```
#check mul_assoc
-- [Semigroup G] is enough

#check inv_mul_cancel
-- needs [Group G]

#check ring
-- tactic works over semiring/ring expressions

example {G : Type*} [Group G] (a b : G) :
  (a * b)-1 = b-1 * a-1 := by
  group
```

读定理类型

- 最弱结构给最大复用
- `[Group G]` 比
`[CommGroup G]` 更一般
- AI 常把结构假设写得过强
- 先 `#check`，再决定变量

更深入：结构迁移比重复证明更稳定

```
def equivProd : PairInt  $\simeq$  Int  $\times$  Int := ...
```

```
instance : CommRing PairInt :=  
  equivProd.commRing
```

```
def equivRat : WrappedQ  $\simeq$   $\mathbb{Q}$  := ...
```

```
instance : Field WrappedQ :=  
  equivRat.field
```

- 复用已有实例，减少手写字段
- 把难点转化为证明 equivalence
- 适合 wrapper 、 subtype 、 同构对象
- AI 可先建议迁移路径，再由 Lean 检查

课堂重点： instance 不一定从零写， Mathlib 常通过等价、同构、子结构复用。

AI 写 Lean 的第一类错：statement 本身错

AI 草稿

```
example (K : Type*) [Field K] (x : K) :  
  x / x = 1 := by  
  field_simp
```

修正版

```
example (K : Type*) [Field K] (x : K) (hx : x ≠ 0) :  
  x / x = 1 := by  
  field_simp [hx]
```

- `x = 0` 是反例
- Lean 不会替你补数学假设
- 失败不是 tactic 问题
- 先修 statement，再修 proof

AI 写 Lean 的第二类错：结构层级选错

AI 草稿

```
example (G : Type*) [Group G] (a b : G) :  
  (a * b)-1 = a-1 * b-1 := by  
  group
```

一般群中的正确命题

```
example (G : Type*) [Group G] (a b : G) :  
  (a * b)-1 = b-1 * a-1 := by  
  group
```

诊断

- 非交换群里逆元顺序反转
- `group` 只能证明真命题
- 原结论需要
 `[CommGroup G]`
- 这就是层级假设的意义

AI 写 Lean 的第三类错：API 幻觉

AI 草稿

```
example : (Finset.range 5).card = 5 := by
  exact Finset.card_range_five
```

修正版：先查真实 API

```
#check Finset.card_range

example : (Finset.range 5).card = 5 := by
  exact Finset.card_range 5
```

- 不存在的名字立即失败
- 不要凭命名风格猜库 API
- 让 AI 给 `#check` 证据
- LeanSearch / Loogle 可辅助搜索

AI 写 Lean 的第四类错：把 decide 当万能证明

AI 草稿

```
example (P : Prop) : P ∨ ¬ P := by
  decide
```

两种修正

```
example (P : Prop) [Decidable P] : P ∨ ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h

example (P : Prop) : P ∨ ¬ P := by
  classical
  exact em P
```

1. `decide` 需要 `[Decidable P]`
2. 有限命题适合 `decide`
3. 一般排中律用 `classical`
4. AI 常混淆计算性与经典逻辑

把 AI 纳入 Lean workflow

阶段	问 AI 要什么	用 Lean 检查什么
Statement	变量、typeclass、必要假设	是否表达原数学命题
Search	`#check` 证据、候选 theorem	名字是否真实，定义是否选对
Proof	短 proof、局部 lemma	是否编译，是否过度依赖脆弱 simp
Review	反例、强弱版本	假设是否过强或过弱

一句话：AI 负责提案，Lean kernel 负责检查，数学家负责语义。

给 AI 的 prompt 要先限制任务

请把下面命题形式化为 Lean 4 / Mathlib 。
先只给 theorem statement ，不要证明。

要求：

1. 列出变量和 typeclass 假设。
2. 说明是否需要非零、有限、可判定等假设。
3. 给出相关 #check 证据。
4. 不要发明 theorem 名字。
5. 如果 statement 可能有多个版本，请给出差异。

- 先对齐对象
- 再对齐结构
- 最后才证明

当前瓶颈不是 Lean 检查不了，而是语义容易偏

问题	表现	解决动作
漏假设	<code>`x / x = 1`</code> 少 <code>`x ≠ 0`</code>	让 AI 先列必要假设
层级错	把 Group 命题写成 CommGroup 才成立	<code>`#check`</code> 定理需要的最弱结构
API 幻觉	不存在的 theorem 名字	要求 <code>`#check`</code> 或检索证据
过度特化	明明可泛化却写成 <code>`ℝ`</code>	读 Mathlib 现有定理类型
证明脆弱	长 tactic 一把梭	拆 lemma，短 proof，局部自动化

更深入：Lean 的可信边界

Lean 会检查

- 形式化后的 theorem statement
- proof term 是否具有该类型
- 所有 instance 是否能被合成
- 定义展开后的等式是否成立

Lean 不会替你检查

- 自然语言原意是否正确
- 选择的 Mathlib 定义是否最合适
- 假设是否过强或过弱
- 结果是否值得作为数学定理

Finset 是组合计数的基本容器

Finset α

- 有限、无重复的元素集合
- ``card`` 表示基数
- ``range n`` 是 ``0, ..., n-1``
- ``filter`` 表示按条件筛选

证明习惯

- ``#check Finset.card_range``
- ``simp`` 处理简单计数
- ``native_decide`` 处理小枚举
- ``sum_range_succ`` 拆最后一项

Finset 小定理：基数、筛选和求和

```
open BigOperators
open Finset

#check Finset.card_range
#check Finset.filter
#check Finset.sum_range_succ

example : (Finset.range 5).card = 5 := by
  exact Finset.card_range 5

example :
  ((Finset.range 6).filter fun n => n % 2 = 0).card = 3 := by
  native_decide

example (n : Nat) :
   $\sum i \in \text{Finset.range } (n + 1), (1 : \text{Nat}) = n + 1 :=$  by
  simp
```

讲法

- 先查 API
- 小枚举用计算关闭
- 求和先看范围
- 复杂计数再做归纳

Finset 小定理：前 n 项求和用归纳拆最后一项

```
theorem sum_id (n : Nat) :  
   $\sum i \in \text{Finset.range } (n + 1), i = n * (n + 1) / 2 :=$  by  
  symm  
  apply Nat.div_eq_of_eq_mul_right (by norm_num : 0 < 2)  
  induction n with  
  | zero =>  
    simp  
  | succ n ih =>  
    rw [Finset.sum_range_succ, mul_add 2, ← ih]  
    ring  
  
example :  $\sum i \in \text{Finset.range } 5, i = 10 :=$  by  
  norm_num [sum_id]
```

证明节奏

1. ``sum_range_succ`` 拆项
2. 归纳假设替换旧和
3. ``ring`` 做代数整理
4. ``norm_num`` 跑具体值

组合计数里常见套路：有限和的结构由 ``Finset`` 管，代数整理交给 `tactic`。

SimpleGraph = 顶点类型 + 邻接命题

对象	Lean 形态	读法
图	<code>`G : SimpleGraph V`</code>	<code>`V`</code> 是顶点类型
边	<code>`G.Adj u v`</code>	<code>`u`</code> 和 <code>`v`</code> 相邻
无向性	<code>`symm`</code> 字段	邻接关系对称
无自环	<code>`loopless`</code> 字段	没有 <code>`G.Adj v v`</code>
有限小图	<code>`V = Fin n`</code>	可以用 <code>`fin_cases`</code> 枚举

图论形式化的第一步不是画图，而是明确顶点类型和邻接谓词。

三点路径图：定义邻接并证明结构字段

```
def path3 : SimpleGraph (Fin 3) where
  Adj i j :=
    (i = 0 ∧ j = 1) ∨
    (i = 1 ∧ j = 0) ∨
    (i = 1 ∧ j = 2) ∨
    (i = 2 ∧ j = 1)
  symm := by
    unfold Symmetric
    intro i j h
    fin_cases i <|> fin_cases j <|> simp_all
  loopless := by
    constructor
    intro i h
    fin_cases i <|> simp_all
```

- 顶点是 `0,1,2`
- 边是 `0-1` 和 `1-2`
- `symm` 证明无向性
- `loopless` 证明无自环

SimpleGraph 小定理：展开定义后交给 simp

```
#check SimpleGraph
#check SimpleGraph.Adj
#check SimpleGraph.ne_of_adj

example : path3.Adj 0 1 := by
  simp [path3]

example : path3.Adj 1 2 := by
  simp [path3]

example : ¬ path3.Adj 0 2 := by
  simp [path3]

example (i : Fin 3) : ¬ path3.Adj i i := by
  fin_cases i <|> simp [path3]

example : (T : SimpleGraph (Fin 3)).Adj 0 1 := by
  decide
```

证明模式

1. 先读 `Adj` 的定义
2. 展开 `path3`
3. 有限顶点可枚举
4. `simp` / `decide` 收尾

SimpleGraph 小定理：邻接自动排除自环

```
example (i j : Fin 3) (h : path3.Adj i j) : i ≠ j := by
  exact SimpleGraph.ne_of_adj path3 h

example : path3.Adj 0 1 ∧ ¬ path3.Adj 0 2 := by
  constructor
  · simp [path3]
  · simp [path3]

example : (T : SimpleGraph (Fin 4)).Adj 0 3 := by
  decide
```

讲法

- ``ne_of_adj`` 来自无自环
- 具体小图展开定义
- 合取目标用 ``constructor``
- 完全图邻接可 ``decide``

图论证明经常在“抽象定理”和“展开具体定义”之间切换。

写 instance 的课堂检查表

1. 先写对象类型：G / R / K / C 是什么？
2. 再写要搜索的结构：AddGroup、Group、CommRing、Field、Module、Category？
3. 用 #print 或错误信息查看还缺哪些字段。
4. 子结构先写 carrier，再补封闭性字段。
5. 数据字段先给定义：add/mul/zero/one/smul/Hom/id/comp。
6. 用 #synth 确认实例存在。
7. 用 #check 读可用定理的最弱假设。
8. 让 AI 写草稿，但让 Lean 和人工语义审核收尾。